## NAME

compare − longest common subsequence of two files

## SYNOPSIS

**compare** [*options*] *file1 file2*

## DESCRIPTION

**compare** computes the Longest Common Subsequence (LCS) of *file1* and *file2*. By default it prints their differences as an edit script; with **−c** it prints the common (matching) records instead, and with **−l** it prints only the LCS length.

Each difference is introduced by a header line:

<<<<< n *file1* [*begin*:*end*] <<<<<
(records present only in *file1*)
>>>>> n *file2* [*begin*:*end*] >>>>>
(records present only in *file2*)

where *n* is the one-based difference number and *[begin:end]* the half-open range of records involved. A final line reports the LCS length, and when compiled with duration reporting enabled, a timing line such as *RickSpace: 28 operations in 9 microseconds @3.111 MHz* follows.

The two files are compared at one of three granularities:

**record** each input line is one record (the default);

**text** (**−t**) the whole file is one sequence of Unicode code points, so multi-byte characters are never split;

**word** (**−w**) each whitespace-separated word is one record.

Three LCS algorithms are available through **−a**:

**d** Hunt−Szymanski (1977), the default: a threshold array over the match lists, O((r+n) log n) time.

**s** RickSpace (Rick 2000): forward/backward contours plus Hirschberg recursion, O(ns) space.

**t** RickTime (Rick 1995): a single diagonal sweep, O(ns + d) space.

## OPTIONS

**−a** *d*|*s*|*t*
Select the LCS algorithm: **d** (Hunt−Szymanski, the default), **s** (RickSpace), or **t** (RickTime). Accepts **−ad** or **−a d**.

**−b** Ignore whitespace: collapse each run of whitespace to a single space before comparing, and expand the reported spans back to the original positions.

**−c** Print the correspondence (the common, matching records) instead of the differences.

**−e** Print a report of the host environment (locale, processor, and memory) and exit without comparing.

**−f** Treat the input files as FASTA and strip header lines; implies **−t**.

**−h** Print a usage synopsis and exit.

**−i** Perform a case-insensitive comparison.

**−j** *join* Coalesce nearby differences that lie within *join* records of each other.

**−l** Report only the LCS length; skip reconstructing the differences.

**−p** *prefix*
Print *prefix* lines of context before each difference.

**−q** Treat *file1* and *file2* as literal strings rather than filenames; implies **−t**.

**−s** *suffix*
Print *suffix* lines of context after each difference.

**−t** Compare the whole file as a sequence of Unicode code points (the text path) rather than line-by-line records.

**−V** Print the version and exit.

**−w** Split each line into whitespace-separated words and compare word by word (record mode).

## EXIT STATUS

**0** Success (including **−h**, **−V**, and **−e**).

**1** Error: a command-line parse failure, a missing file, or an algorithm failure.

## ENVIRONMENT

The locale (in particular **LANG**, **LC_ALL**, and **LC_NUMERIC**) controls the thousands separator used in the reported counts and the locale name shown by **−e**. On a system whose locale defines digit grouping, counts are printed with comma separators (for example **LCS Length = 4,591,960**); otherwise they are printed ungrouped.

## EXAMPLES

Compare two files line by line and show the differences:

```
compare old.txt new.txt
```

Compare as text (code points), case-insensitively, showing only the length:

```
compare −t −i −l a.fa b.fa
```

Compare two FASTA genomes with the linear-space algorithm and show matches:

```
compare −as −c −f ecoli.seq ecoli_100subs.seq
```

Compare two literal strings word by word:

```
compare −q −w "the quick fox" "the lazy fox"
```

## SEE ALSO

**delta**(1), **diff**(1), **cmp**(1)

The LCS algorithms and their performance bounds are described in *LCSRickSpace.md* and *LCSRick-Time.md* in the project documentation.